

CampusConnect

Campus Social Network — Requirements, Architecture & Proof of Concept

Assignment 1 Report — Omar Ibrahim, Salama Aldhaheri, Makar Ulesov, Aro Dana

MBZUAI — generated from docs/ in this repository

Stakeholder Analysis

Lead: Salama (1.1). Drafted by the team 2026-09-29, informed by Omar's earlier individual CampusPulse lab exercise (different product, same course — reused as reference only, not as team history; see repo commit `29b5afc`).

Stakeholder types: end user, operations, business, regulator, negative stakeholder.

S1 — Student

- Type: End user
- Goal: Discover and share campus updates in one place; comment and engage freely.
- Concern: Over-moderation/censorship of legitimate posts; loss of privacy.
- **Conflict:** wants maximum openness and speed of publishing — directly in tension with S3 (moderator) and S4 (administration), who want review and control before/after publishing.

S2 — Group officer

- Type: End user (elevated — publishes on behalf of a verified group)
- Goal: Publish and hand off drafts quickly, reach the right audience (US-4, ADR-04).
- Concern: Approval friction or moderation delay makes the group look unresponsive.
- **Conflict:** wants fast, low-friction publishing — in tension with S3/S4's need for review before content reaches a wide audience.

S3 — Campus moderator

- Type: Operations
- Goal: Act quickly on reports, retain evidence and a decision record for appeals (ADR-01).
- Concern: Losing evidence, untraceable decisions, unauthorised officer publishing.
- **Conflict:** wants control and an auditable process — in tension with S1/S2's desire for speed and minimal friction.

S4 — University administration / Student Affairs

- Type: Business / regulator
- Goal: Platform stays compliant (official group verification, no impersonation), no PR incidents traceable to the university.
- Concern: Unverified groups, harmful content reaching students under the university's name.
- **Conflict:** wants verification/compliance overhead — in tension with S2's desire for fast onboarding of new groups, and with S1's expectation of open access.

S5 — Bad actor / spammer (negative stakeholder)

- Type: Negative stakeholder
- Goal (adversarial): Post spam, impersonate a group, or repeatedly republish removed content.

- Concern (system's, not theirs): Must be mitigated by moderation (ADR-01) and account/group verification (S4), without over-restricting legitimate users (S1/S2).

Cross-stakeholder conflict summary

Tension	Sides	How the architecture addresses it
Speed vs. control	S1/S2 vs. S3/S4	Publish-then-moderate (not pre-approval) — posts go live immediately, moderators act after a report (ADR-01)
Openness vs. compliance	S1 vs. S4	Group verification badge (S4) without gating individual student posts
Officer autonomy vs. auditability	S2 vs. S3	Draft hand-off is versioned (ADR-04); moderation decisions are recorded, not silent deletes (ADR-01)

Project Scope

Status: agreed by Omar, Salama, Makar (2026-09-29). Aro to review 1.4–1.5 against this. Salama leads 1.1–1.3 and owns further edits to this file.

Product name

CampusConnect

Problem

Students at MBZUAI don't have a single place to share campus-specific updates (events, club posts, study groups, lost & found) — general social media is too broad and noisy, and official channels (email, LMS announcements) are one-way and slow.

First-release scope (Assignment 2 target)

- Student accounts, campus-verified (e.g. university email)
- Create/view text posts on a shared campus feed
- Comment on posts
- Basic visibility control (public to campus vs. specific groups/clubs)
- Moderation: campus moderators can hide/remove any post (report → hide → decision record, ADR-01); group officers can additionally remove posts within their own group
- AI feature: summarize long comment threads into a short digest (ADR-02, owner: Aro)

Future features (out of scope for A1/A2, mentioned for context only)

- Real-time notifications (WebSockets)
- Rich text / media-heavy posts (React rich text editor)
- Direct messaging
- Events/calendar integration

Explicitly excluded (this proof of concept, Assignment 1)

- No real authentication (mock/fictional users)
- No persistent database (in-memory only)
- No AI integration wired up (planned in architecture only)
- No production-grade styling/UX

Open questions for the team

- Appeals process for moderation decisions (Salama/ADR-01)
- Aro to confirm this scope against 1.4–1.5 user stories and traceability matrix

Functional Requirements

Lead: Salama (1.2). MoSCoW-prioritized, traced to user stories in `user-stories.md`.

In scope (Assignment 2 release)

ID	Requirement	Priority	Source	Story
FR-1	A student can create a text post to a campus/group feed	Must	S1, S2	US-1
FR-2	A student can view the campus feed, newest posts first	Must	S1	US-2
FR-3	A student can comment on a post	Must	S1	—
FR-4	A post's audience can be set to campus-wide or a specific group	Must	S1, S2	— (ADR-01)
FR-5	A campus moderator can hide a reported post and record a decision	Must	S3	US-3 (ADR-01)
FR-6	A group officer can remove a post within their own group	Must	S2, S3	US-4 (ADR-01)
FR-7	A group officer can hand a draft post to another officer before publishing	Should	S2	ADR-04
FR-8	The system generates a short summary of a long comment thread on request	Should	S1	ADR-02
FR-9	A group can be marked "verified" by an authority (Student Affairs)	Should	S4	—
FR-10	A student can report a post for moderator review	Must	S1, S3	US-3

Explicitly out of scope (Won't — this project)

ID	Excluded	Reason
EX-1	Native mobile apps	Browser-only release; see scope.md
EX-2	Direct messages	Not core to the campus-feed use case
EX-3	Non-university/external users	University-only by design (FR-9 verification)
EX-4	Payments	Out of scope for a social feed
EX-5	Video hosting	Text/rich-text posts only (React rich text, A2)
EX-6	Real (non-mocked) authentication in Assignment 1 POC	POC demonstrates the create-post flow only; see README

Non-Functional Requirements

Lead: Salama (1.3), architectural drivers ranked with technical input from Makar and Omar (2.1).
Each target is measurable — a vague "fast" or "secure" is not acceptable here.

ID	Category	Requirement	Measurable target	Source / driver
NFR-1	Performance	Feed loads quickly under normal load	95% of <code>GET /posts</code> requests return in < 500ms at up to 200 concurrent users	S1 (usability)
NFR-2	Scalability	Supports a pilot campus population	5,000 students, 200 groups, without redesign (informed by earlier individual CampusPulse analysis)	S4
NFR-3	Availability	Usable during class hours	99% uptime, 8am–10pm campus time, during Assignment 2 demo period	S1, S4
NFR-4	Moderation SLA	Reports are actionable quickly enough to limit harm	A report is visible to a moderator within 1 minute (near-real-time via WebSocket, ADR-04 infra)	S3
NFR-5	Data integrity	Moderation decisions are never silently lost	100% of hide/remove actions produce a durable <code>MODERATION_RECORD</code> row (ADR-01, ADR-03) — verified by test, not just code review	S3
NFR-6	Accessibility	Usable with assistive tech	WCAG 2.1 AA on the core feed/post/comment flows	S1
NFR-7	Security (A2)	Only verified university accounts can post	100% of posts have a non-null <code>author_id</code> linked to a university-verified account; POC/Assignment 1 explicitly mocks this (see README)	S4
NFR-8	Concurrency correctness	No silent draft overwrite between two officers	Version-conflict rate observable and rejected writes return 409, not silent overwrite (ADR-04) — verified by test in Assignment 2	S2, S3

Architectural drivers ranking (2.1)

Ranked by the team, with technical feasibility input from Makar and Omar:

1. **Data integrity (NFR-5)** — moderation records must never silently disappear; drives PostgreSQL choice (ADR-03) over a less strict store.
2. **Concurrency correctness (NFR-8)** — draft hand-off must not lose officer work; drives optimistic-concurrency + WebSocket design (ADR-04).
3. **Performance (NFR-1)** — feed responsiveness is what students actually notice; drives keeping the POC/A2 read path simple (no heavy joins on the hot path).
4. **Scalability (NFR-2)** and **Availability (NFR-3)** — real but secondary at pilot scale; informs choosing Postgres (scales further than SQLite) without over-engineering for the POC/early Assignment 2 stage.
5. **Accessibility (NFR-6)** and **Security (NFR-7)** — required for release, but not decided by an ADR in this assignment; flagged as Assignment 2 implementation requirements.

User Stories & Acceptance Criteria

Lead: Aro. Drafted together by the team (2026-09-29) from the agreed scope; Aro to refine wording, add assumptions/open questions, and own the traceability matrix going forward.

US-1: Create a post

As a student, I want to post an update to the campus feed, so that others can see it.

Acceptance criteria: - Given I am on the campus feed, when I submit a post with an author name and content (1–1000 chars), then the post appears at the top of the feed within the same page load. - Given I submit a post with empty content, when I hit submit, then I see a validation error and no post is created (matches `POST /posts` 422 response).

US-2: View the campus feed

As a student, I want to see recent campus posts, so that I stay informed.

Acceptance criteria: - Given posts exist, when I open the feed, then I see them ordered newest first. - Given no posts exist, when I open the feed, then I see an empty feed (no error).

US-3: Moderator hides a reported post

As a campus moderator, I want to hide a reported post, so that harmful content stops spreading.

Acceptance criteria: - Given a post has been reported, when a moderator hides it, then it disappears from the public feed but the content, reporter, and decision are retained for appeal review. - Given a post is hidden, when a non-moderator views the feed, then the post is not shown.

US-4: Group officer moderates their own group

As a group officer, I want to remove a post in my own group, so that I can moderate my community.

Acceptance criteria: - Given a post belongs to my group, when I remove it, then it disappears from that group's feed. - Given a post belongs to a group I don't officer, when I try to remove it, then I am denied.

US-5: Comment on a post

As a student, I want to comment on a post, so that I can discuss it with others.

Acceptance criteria: - Given a post exists, when I submit a comment (1–1000 chars), then it appears under the post, oldest first, without reloading the page. - Given I submit an empty comment, when I hit submit, then I see a validation error and no comment is created (same validation shape as `POST /posts`, see `api-contract.md`).

US-6: Set a post's audience

As a student or group officer, I want to choose whether a post is campus-wide or group-only, so that I control who sees it.

Acceptance criteria: - Given I am creating a post, when I pick "campus-wide" or "group-only", then the post is only visible to that audience when others view the feed. - Given a post is group-only, when a student who isn't a member of that group views the campus feed, then the post does not appear.

US-7: Student Affairs verifies a group

As a Student Affairs staff member, I want to mark a group as verified, so that students can trust its official badge (S4, NFR-7).

Acceptance criteria: - Given an unverified group, when Student Affairs marks it verified, then its posts show a "verified" badge in the feed. - Given a group is not verified, when a student views its posts, then no verified badge is shown, and the group is visually distinguishable from verified ones.

Assumptions

- "Group" membership and officer role are out of scope for the Assignment 1 POC (mocked/fictional).
- Moderation (US-3, US-4) and verification (US-7) are architecture-only for A1; not implemented in the POC backend.
- US-7's "Student Affairs staff member" role and its access control are not detailed further here — out of this assignment's depth, flagged for Assignment 2 auth design.

Open questions

- Appeals process detail for US-3 (linked to ADR-01, still open).
- Exact reporting flow (who can report, is it rate-limited) — not yet specified.
- US-7: who grants Student Affairs staff their role in the system itself (bootstrapping problem) — not specified.

Traceability matrix

Full requirement definitions: `functional-requirements.md`, `non-functional-requirements.md`.

Stakeholder sources: `stakeholders.md`.

Story	Functional req.	NFR	ADR	POC demo / test	Status
US-1 Create post	FR-1	NFR-1	—	<code>POST /posts</code> , frontend form, <code>test_create_post</code>	Implemented in POC
US-2 View feed	FR-2	NFR-1	—	<code>GET /posts</code> , frontend list, <code>test_list_posts_newest_first</code>	Implemented in POC
US-3 Moderator hides post	FR-5, FR-10	NFR-4, NFR-5	ADR-01	not in POC	Planned for A2
US-4 Officer moderates group	FR-6	NFR-5	ADR-01	not in POC	Planned for A2

Story	Functional req.	NFR	ADR	POC demo / test	Status
(unstoried) Draft hand-off	FR-7	NFR-8	ADR-04	not in POC	Planned for A2
(unstoried) Thread summary	FR-8	—	ADR-02	not in POC	Planned for A2
(unstoried) Data persistence	—	NFR-2, NFR-5, NFR-7	ADR-03	not in POC	Planned for A2
US-5 Comment on post	FR-3	NFR-1	—	not in POC	Planned for A2
US-6 Set post audience	FR-4	—	ADR-01	not in POC	Planned for A2
US-7 Verify a group	FR-9	NFR-7	—	not in POC	Planned for A2

All FR/NFR now have at least one story or an explicit "unstoried" architecture-only row. FR-9 still has no owning ADR (see `requirements-review.md` open questions) — Salama to decide whether it needs its own ADR or folds into ADR-01's scope.

Requirements Review (1.5)

Reviewed together 2026-09-29 by Omar, Salama, Makar (Aro joined for user stories, see `user-stories.md`). Feasibility checked by Makar and Omar against the POC and A2 architecture. Re-reviewed 2026-10-05: appeals process and FR-9 ownership resolved, see below.

Review notes

- FR-1/FR-2 (create/view post) are implemented and tested in the Assignment 1 POC (`backend/test_main.py`) — the only requirements with working code so far.
- FR-5/FR-6 (moderation) depend on ADR-01's decision (moderators + group officers) — checked against the stakeholder conflict table in `stakeholders.md`, no contradiction found.
- FR-7 (draft hand-off) and NFR-8 (concurrency) are consistent with ADR-04's decision (optimistic concurrency + WebSocket presence) — Makar/Omar confirmed this is buildable with FastAPI WebSockets in the Assignment 2 timeframe (see `project-management/README.md`).
- FR-9 (group verification) now folds into ADR-01's policy (updated 2026-10-05) rather than getting its own ADR — Student Affairs role grants the badge; no separate decision needed.
- NFR-2's "5,000 students / 200 groups" figure is a placeholder pilot target, not a validated number from Student Affairs — flagged as an assumption below.
- Appeals process (resolved 2026-10-05, ADR-01): author requests review, a second moderator decides. This assumes ≥ 2 active moderators — now tracked as a risk, see `project-management/README.md` R6.

Assumptions

- University context is MBZUAI; "campus-verified" means a university email domain check (mocked in the POC, real integration is Assignment 2 scope).
- The 5,000-student / 200-group scale (NFR-2) is an assumed pilot size, not confirmed by any real stakeholder at MBZUAI.
- Group verification (FR-9) is performed by a human authority (Student Affairs), not automated — process itself is out of scope for this assignment.
- Appeals review assumes at least 2 active campus moderators exist at any time (ADR-01) — not validated against real staffing.

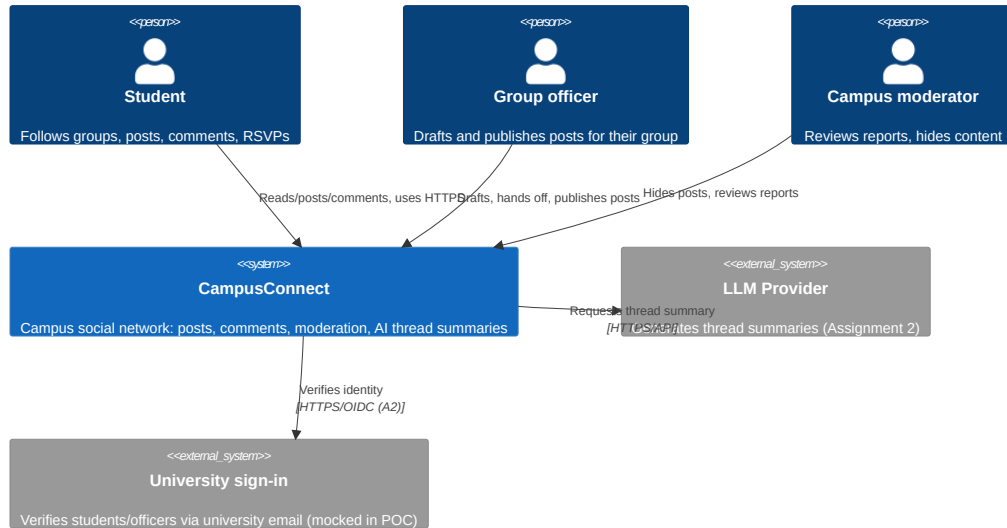
Open questions

- Whether NFR-3's uptime target needs to hold during the 3-minute demo only, or continuously through Assignment 2 grading — assumed continuous for now.
- Who grants the Student Affairs role itself in the system (bootstrapping problem, carried over from `user-stories.md` US-7) — not specified.

C4 — Level 1: System Context

Editable source (Mermaid) — renders directly on GitHub.

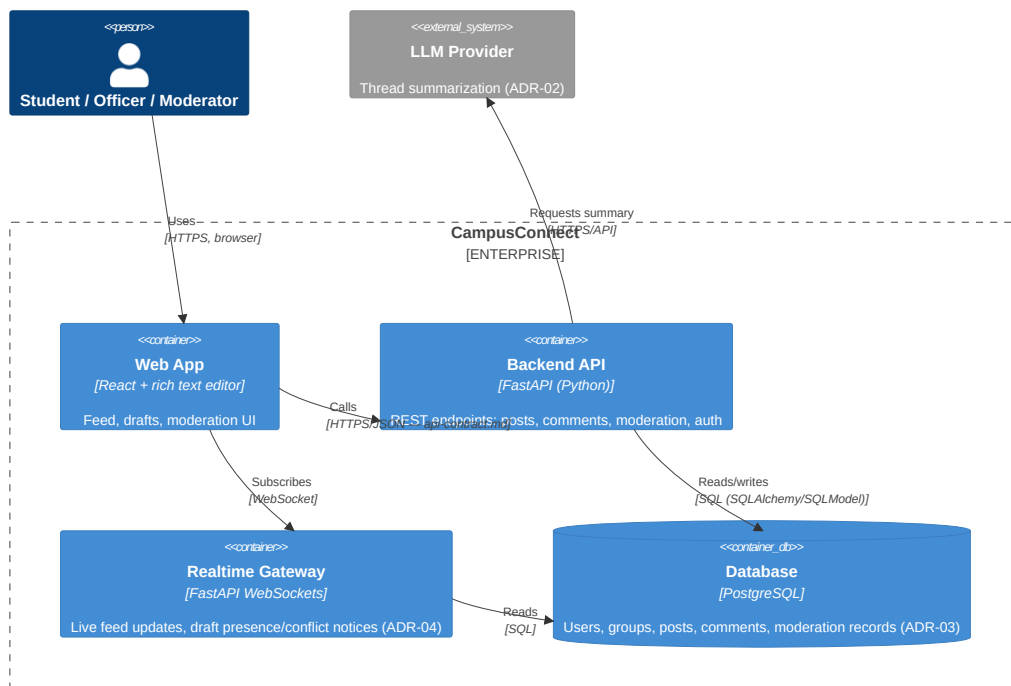
CampusConnect — System Context



C4 — Level 2: Containers

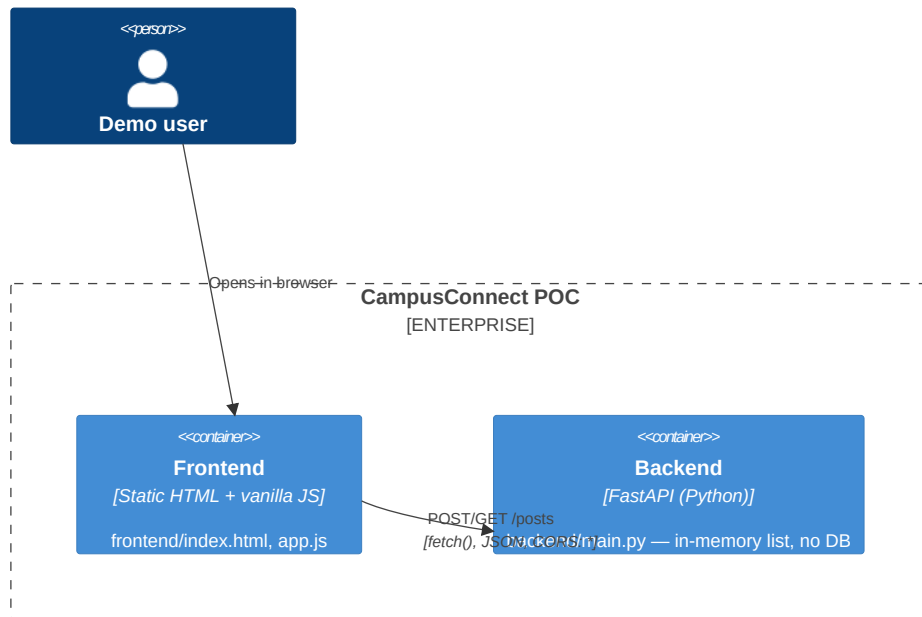
Assignment 2 target architecture. The Assignment 1 POC only implements the Browser SPA (plain HTML/JS here, not React yet) and the API container (in-memory, not Postgres) — see README "POC simplifications".

CampusConnect — Containers (Assignment 2 target)



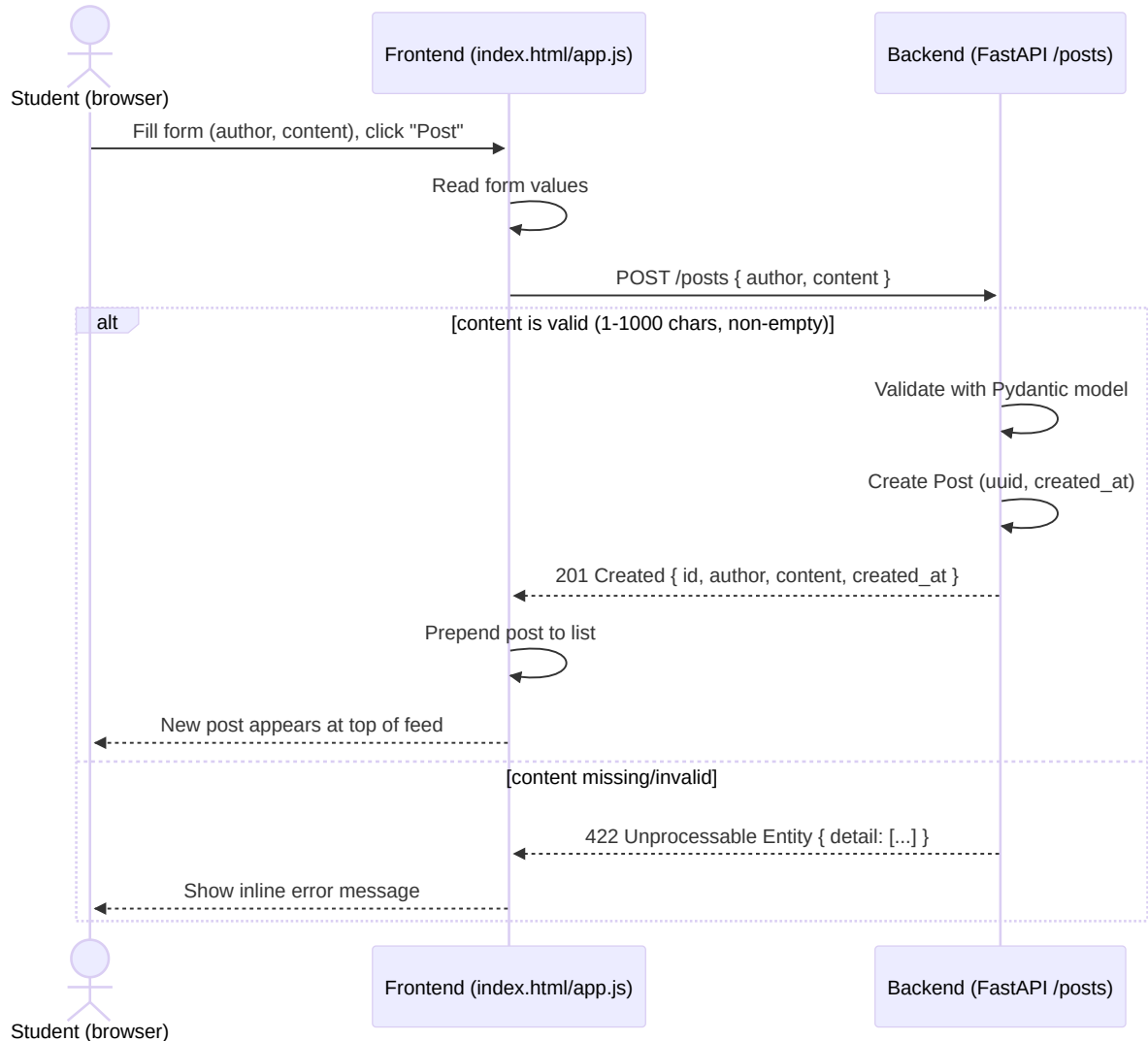
Assignment 1 POC container view (what actually runs today)

CampusConnect POC — Containers (Assignment 1, as implemented)



Collaboration Sequence Diagram — Create a Post (POC demo flow)

Matches the actual POC implementation and `api-contract.md` exactly — this is the sequence the 3-minute demo walks through.



API Contract — Proof of Concept

Agreed by Omar & Makar (2026-09-29). Must match `backend/main.py` and the demo exactly.

POST /posts

Create a post.

Request

```
{
  "author": "string, 1-100 chars, required",
  "content": "string, 1-1000 chars, required"
}
```

Response — 201 Created

```
{
  "id": "uuid string",
  "author": "string",
  "content": "string",
  "created_at": "ISO 8601 UTC timestamp"
}
```

Response — 422 Unprocessable Entity (validation error, FastAPI/Pydantic v2 default shape)

```
{
  "detail": [
    {
      "type": "missing",
      "loc": ["body", "content"],
      "msg": "Field required",
      "input": { "author": "Omar" }
    }
  ]
}
```

GET /posts

List posts, newest first.

Response — 200 OK

```
[
  {
    "id": "uuid string",
    "author": "string",
    "content": "string",
  }
]
```

```
    "created_at": "ISO 8601 UTC timestamp"
  }
]
```

GET /health

Response — 200 OK

```
{ "status": "ok" }
```

Notes

- No auth in the POC — `author` is free text, not a verified identity.
- No persistence — restarting the backend clears all posts.
- CORS is open (`*`) for local demo only, not a production setting.

Repository Structure (2.5)

Owners: Makar (backend), Omar (frontend). This documents the actual repo layout as of the Assignment 1 POC, plus what Assignment 2 adds.

Current (Assignment 1 POC)

```

campus-social-network/
├── README.md                setup/run instructions, team, POC simplifications
├── backend/
│   ├── main.py             FastAPI app: POST/GET /posts, GET /health
│   ├── test_main.py        pytest suite (4 tests, run: pytest -q)
│   └── requirements.txt     fastapi, uvicorn, pytest, httpx
├── frontend/
│   ├── index.html          post form + feed
│   └── app.js              fetch calls to the backend, matches api-contract.md
├── docs/
│   ├── requirements/
│   │   ├── scope.md        1.1 product scope
│   │   ├── stakeholders.md 1.1 stakeholder analysis + conflicts
│   │   ├── functional-requirements.md 1.2
│   │   ├── non-functional-requirements.md 1.3 (+ 2.1 driver ranking)
│   │   ├── user-stories.md 1.4-1.5, traceability matrix
│   │   └── requirements-review.md 1.5
│   ├── architecture/
│   │   ├── api-contract.md 2.4 interface contract (POC)
│   │   ├── repository-structure.md 2.5 (this file)
│   │   ├── adr/            2.7 – adr-01..adr-04
│   │   └── diagrams/       2.2-2.3, 2.6 – C4, sequence, ER (Mermaid)
│   └── project-management/
│       └── README.md        Section 3

```

Assignment 2 target (planned, not yet built)

```

campus-social-network/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   └── models/      SQLAlchemy models (User, Group, Post,
Comment,
│   │   │               ModerationRecord, Draft – see er-diagram.md)
│   │   ├── routers/    posts.py, comments.py, moderation.py, ws.py
│   │   ├── services/   thread-summarization client (ADR-02)
│   │   └── db.py
│   ├── alembic/         schema migrations (ADR-03)
│   └── tests/

```

```

└─ frontend/
  └─ src/
    └─ components/      React components (feed, post form w/ rich text,
moderation UI)
    └─ api/             typed client for the backend contract
    └─ package.json
└─ docs/               same structure as A1, sections filled in further

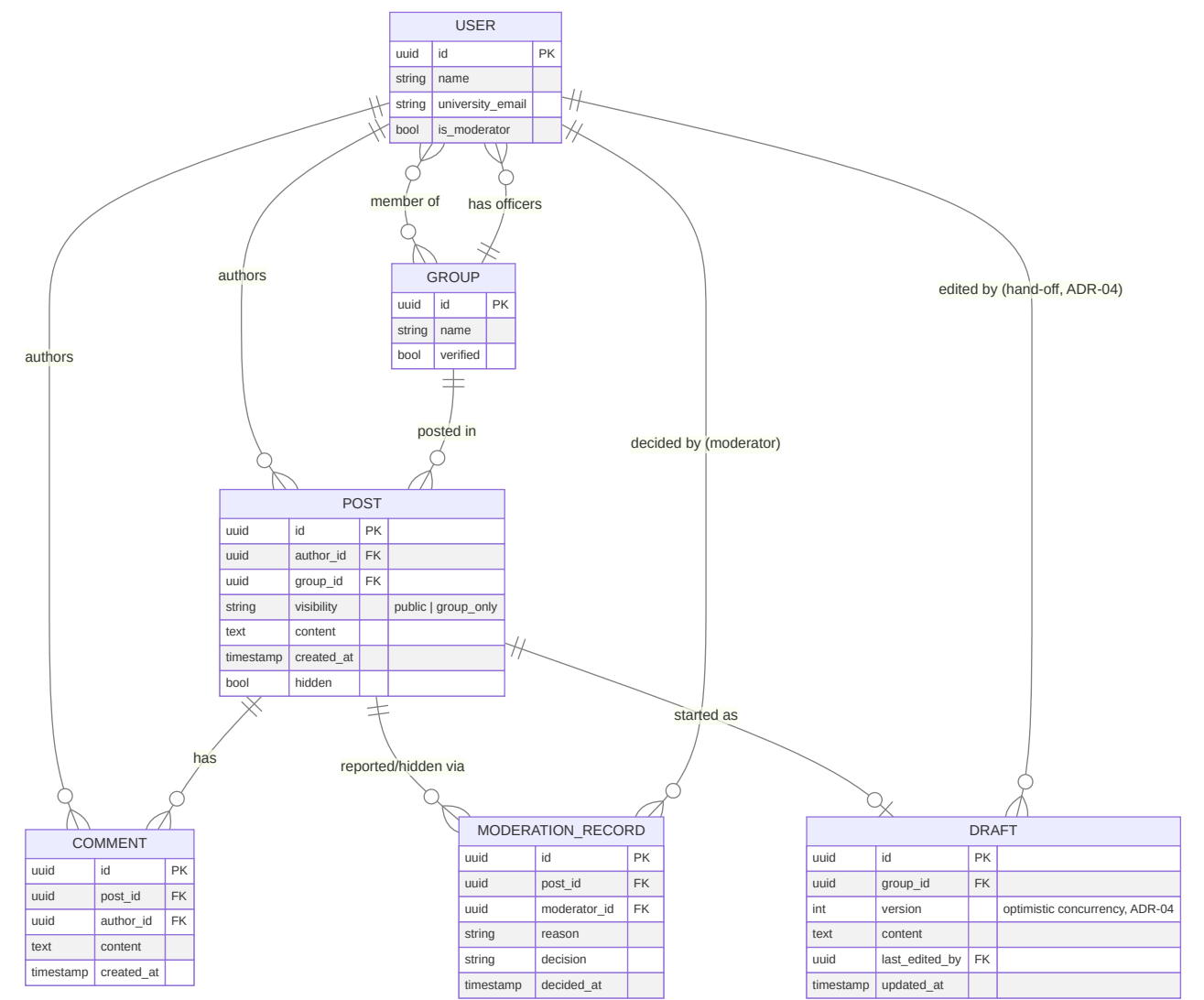
```

Rationale

- `backend/` and `frontend/` are separate top-level dirs (not a monorepo tool like Nx/Turborepo) — team is 4 people, 2 assignments; that tooling overhead isn't justified yet.
- `docs/` mirrors the assignment's own section numbering (1.x requirements, 2.x architecture, 3.x project management) so any grader — or teammate — can find a section without guessing.
- Assignment 2's `backend/app/` split (models/routers/services) follows ADR-03 (relational data needs real model classes) and ADR-04 (concurrency logic lives in a service, not scattered across route handlers).

ER Diagram

Assignment 2 target data model (PostgreSQL, ADR-03). The Assignment 1 POC only has an in-memory `Post` shape (`id`, `author`, `content`, `created_at`) — see `api-contract.md`.



ADR-01: Audience Visibility and Access-Control Policy

- Status: draft (owner Salama — final wording still to review)
- Owner: Salama
- Date: 2026-10-05

Context

Who can see a post, who can restrict its audience, and what visibility levels does the platform need? Also covers: audience setting at post-creation (US-6/FR-4), group verification badges (US-7/FR-9), and the appeals process for moderation decisions (US-3).

Visibility levels

- **Campus-wide** — visible to every verified student (default).
- **Group-only** — visible only to members of the posting group (US-6).
- No "private"/DM-style visibility — explicitly out of scope (see `scope.md` exclusions).

Alternatives considered — moderation authority

1. Moderators only (centralized, slower, consistent)
2. Moderators + group officers for their own group (chosen — faster response, some inconsistency risk)
3. Fully decentralized (any group member can remove) — rejected, too easy to abuse

Alternatives considered — appeals process

1. No appeals (moderator decision is final) — rejected: no recourse for a wrongly hidden post, conflicts with S1's trust in the platform.
2. **Appeal to a second moderator, decision logged** (chosen) — the original author can request review; a different moderator (not the one who hid it) reviews the retained content + original decision record (`MODERATION_RECORD` , ADR-03) and either restores or upholds. Keeps a second set of eyes without needing a full committee process.
3. Committee/admin panel review — rejected for Assignment 2 scope, too heavy for pilot scale (NFR-2: 5,000 students).

Decision

- Moderation authority: moderators + group officers for their own group.
- Appeals: author can request review; a second moderator (not the original) decides, using the retained content and decision record. The appeal outcome is itself recorded.
- Group verification (US-7/FR-9): a Student Affairs role (separate from "moderator") grants the verified badge; this ADR owns the policy, FR-9's exact UI/workflow is Assignment 2 implementation detail, not a separate ADR (per `user-stories.md` traceability note).

Consequences

- MODERATION_RECORD (ER diagram) needs an appeal_status and appeal_decided_by field, distinct from the original moderator_id — a schema detail Makar should carry into ADR-03's implementation, not just this ADR's text.
- Two-moderator appeal review assumes campus has ≥ 2 active moderators at any time — a staffing assumption, not a technical one; flagged here so project management (risks) can track it if it turns out false at pilot scale.
- Officers' power is scoped to their own group only — an officer cannot moderate another group's posts, limiting blast radius of a compromised/malicious officer account.

ADR-02: AI Feature, Data Used, and Output Verification

- Status: draft
- Owner: Aro
- Date: TBD

Context

Team agreed (2026-09-29): thread summarization — an LLM condenses a long comment thread into a short digest. Aro to write up the full rationale, data used, and how a user verifies the summary against the original thread.

Alternatives considered

1. Thread summarization (chosen — clear, demoable, low risk if wrong)
2. Spam/toxicity flagging (rejected for this ADR — overlaps with moderation policy, ADR-01)
- 3.

Decision

Thread summarization. Aro to detail: what triggers it, which model/data, and the UX for a user to check the summary against the real thread (e.g. "view original" link, confidence note).

Consequences

(Trade-offs, risks, what this enables/blocks for later assignments.)

ADR-03: Data Storage Approach

- Status: draft (owner Makar — final wording still to review)
- Owner: Makar
- Date: 2026-09-29

Context

The Assignment 1 POC keeps posts in memory (see README, "explicitly excluded"). Assignment 2 needs real persistence for: users, groups, posts, comments, RSVPs/attendance, and moderation records (report → hide → decision, ADR-01) that must survive restarts and support appeals. Entities are clearly relational: a post belongs to a group and an author, a moderation record references a post and a moderator, a comment references a post and an author.

Alternatives considered

1. **Document store (e.g. MongoDB)** — flexible schema, easy to start, but the data is inherently relational (foreign keys everywhere: post → group, post → author, moderation record → post → moderator) and we'd rebuild joins/consistency checks in application code.
2. **SQLite** — zero ops, fine for the POC, but not realistic for Assignment 2's multi-user, concurrent-write scenario (ADR-04) or for demonstrating a production-plausible architecture.
3. **PostgreSQL (relational)** — native foreign keys and constraints match the data model directly; transactions give ACID guarantees for moderation decision records (can't lose a decision mid-write); mature FastAPI support (SQLAlchemy/SQLModel + Alembic migrations).

Decision

Option 3: PostgreSQL, accessed via SQLAlchemy/SQLModel from FastAPI, with Alembic for schema migrations. Not implemented in the Assignment 1 POC (in-memory only, see README); this ADR governs the Assignment 2 backend.

Consequences

- Requires running/hosting a Postgres instance (local Docker container for dev, per Assignment 2).
- Relational constraints (foreign keys, NOT NULL, unique) enforce data integrity that would otherwise be re-implemented manually in a document store.
- Schema changes need migrations (Alembic) — more process than a schemaless store, but this is the right trade-off given moderation/appeal records must not silently lose fields.
- Sets the ER diagram (2.6) and repository structure (2.5) to be table/model-based, not document-collection-based — Makar to keep this consistent across those sections.

ADR-04: Collaboration and Concurrent Editing Approach

- Status: draft (owner Omar — final wording still to review)
- Owner: Omar
- Date: 2026-09-29

Context

Group officers hand off draft posts to each other before publishing (US-4, scope.md). Two officers may open the same draft around the same time. Assignment 2 must also account for WebSockets and a React rich-text editor, so this decision should not paint us into a corner that can't grow into live co-editing later.

Alternatives considered

1. **Pessimistic locking** — one officer "checks out" a draft; others get a read-only view until it's released. Simple, but a forgotten checkout blocks the whole group, and it gives no live-presence signal (who else is looking at this draft right now).
2. **Real-time collaborative editing** (WebSocket-synced, CRDT/OT, Google-Docs-style) — best UX, but full CRDT/OT is significant scope for a two-assignment project and overkill for the actual use case (handing off a draft, not simultaneous live typing by many people).
3. **Optimistic concurrency with a version field** — each draft has a `version` integer; saving/publishing requires the client's last-seen version to still match, otherwise the save is rejected with a 409 and the officer sees a conflict warning with the latest content to merge manually. A WebSocket channel broadcasts "draft updated" / "officer X is viewing" presence events so conflicts are rare in practice, without needing full CRDT merge logic.

Decision

Option 3: optimistic concurrency (version-checked writes) plus a WebSocket presence/notify channel. This is implemented in Assignment 2, not in the Assignment 1 POC (which has no draft hand-off at all — POC posts are single-author, published immediately, see scope.md "Explicitly excluded").

Consequences

- Officers rarely lose work: conflicts are detected, not silently overwritten.
- Requires the backend to store a `version` per draft and reject stale writes (409) — a real API contract detail to nail down in Assignment 2, not just a UI nicety.
- Simpler to build and reason about than true CRDT-based co-editing; trade-off is that two officers cannot literally type in the same draft at the same character position at the same time — acceptable for this use case (hand-off, not live pair-writing).
- The WebSocket presence channel is reusable for other Assignment 2 features (e.g. "X is typing" on comments), so this isn't throwaway infrastructure.

Project Management

Lead: Aro. Drafted by the team 2026-09-29, based on this repo's actual Git workflow and the assignment brief. Aro owns final wording and risk assessment.

Responsibilities

Member	Lead role
Salama	Requirements & product scope (1.1–1.3, 2.1)
Aro	Planning, validation, report (1.4–1.5, Section 3)
Makar	Backend & data architecture (2.4–2.6)
Omar	Frontend & integration (2.2–2.3)

Each person writes their own lead sections; Aro assembles the final report (per assignment brief).

Git workflow

- Single shared repo: `omar07ibrahim/campus-social-network` (private, teaching team added as a read collaborator).
- `master` is the working branch for Assignment 1 (small team, short timeline); Assignment 2 moves to feature branches + PR review once the codebase is large enough to need it (see risk R2 below).
- Commit convention: imperative subject line, author set to whoever actually wrote the content (`Co-authored-by:` trailer when more than one person contributed to a commit).

Development approach

Lightweight, milestone-driven — not full Scrum (team is 4 people, project is 2 assignments, not a multi-sprint product). Work is grouped by assignment deliverable (requirements → architecture → PM → POC), agreed together at the start (Step 1–3 of the assignment's suggested order), then built in parallel with a review pass before submission.

Risks

Risk	Likelihood	Impact	Mitigation
R1: Requirements/architecture drift apart (docs say one thing, code does another)	Medium	High	<code>api-contract.md</code> is the single source of truth for request/response shape; both sides tested against it (see <code>backend/test_main.py</code>)
R2: <code>master</code> -only workflow breaks down once Assignment 2 has real concurrent backend/frontend work	Medium	Medium	Move to feature branches + PR review at the start of Assignment 2, before the codebase grows
R3: One member (Aro) blocked on the others' sections for the final report	Low	High	Each lead section is its own file under <code>docs/</code> , written independently; Aro assembles, doesn't

Risk	Likelihood	Impact	Mitigation
			wait to start until everyone else finishes
R4: AI feature (thread summarization, ADR-02) scoped too ambitiously for Assignment 2's timeline	Medium	Medium	POC explicitly excludes AI integration (see README); Assignment 2 can start with a stubbed/mockd summary before wiring a real LLM call
R5: Oral exam — a member can't explain a part they didn't personally write	Medium	High	Step 6 of the assignment brief: practice session where each member explains every section, not just their own
R6: Appeals process (ADR-01) assumes ≥ 2 active campus moderators; if only 1 exists, appeals can't be reviewed by a second person	Low	Medium	Flagged in <code>requirements-review.md</code> ; Assignment 2 can fall back to admin review if moderator pool is 1
R7: Two collaborator invites (Makar, Teaching Team/prof) still pending acceptance on GitHub as of 2026-10-05	Medium	High	Follow up directly — repo access for the teaching team is a hard submission requirement (final checklist)

Assignment 2 timeline

(Placeholder milestones — Aro/team to confirm real dates against the course schedule.)

Milestone	Target
Data model + Postgres schema live (ADR-03)	Week 1
FastAPI backend: auth (mocked), posts, comments, moderation	Week 2–3
React frontend, rich text editor	Week 2–3 (parallel with backend)
WebSocket live updates + draft-editing concurrency (ADR-04)	Week 4
AI feature: thread summarization (ADR-02)	Week 4–5
Integration testing, demo prep, report assembly	Week 5–6

Demo Script (≤ 3 minutes)

Lead: Omar (setup + demo prep, per assignment brief).

Before the demo

1. `cd backend && source .venv/bin/activate && uvicorn main:app --reload --port 8000`
2. `cd frontend && python3 -m http.server 5500`
3. Open `http://localhost:5500` in a browser, `http://localhost:8000/docs` in another tab (optional, shows the live API contract).

Script (target: under 3 minutes)

Time	Action	What it shows
0:00–0:20	State the problem in one sentence (from <code>scope.md</code>)	Grounds the demo in the requirement, not just "look, code"
0:20–0:40	Show the empty feed at <code>http://localhost:5500</code>	<code>GET /posts</code> on load, matches <code>api-contract.md</code>
0:40–1:10	Fill the form, submit a valid post	<code>POST /posts</code> → 201 → post appears at top — full flow: browser form → backend validation → response → display (assignment's required demo flow)
1:10–1:30	Submit an empty/invalid post	422 validation error shown inline — contract's error case, not just the happy path
1:30–1:50	Open <code>http://localhost:8000/docs</code> briefly	Shows the contract is live/enforced by the backend, not just described in a doc
1:50–2:30	One sentence each on what's <i>not</i> in the POC and why (auth, DB, moderation, AI)	Sets up the Assignment 2 pitch — ties to ADR-01/02/03/04
2:30–3:00	One sentence on Assignment 2 direction (React, WebSockets, Postgres, LLM summary)	Closes on the architecture, not just the POC

Fallback if live demo breaks

- `backend/test_main.py` passing (`pytest -q`) is the backup evidence the contract holds, screenshot it before the exam if live demo access is uncertain.